Essay #6

Object Detection, Semantic Segmentation, Face Recognition, and Neural Style Transfer

AI Application – The Sixth Online Video Lecture (Convolutional Neural Networks, Weeks 3–4)

Name: _____ Student ID: _____

Introduction

The first two weeks of the Convolutional Neural Networks course showed how a convnet can decide what is in an image. Weeks 3 and 4 move on to harder and more practical questions: where the objects are, which pixel belongs to which object, whose face is in the picture, and even how to paint a new picture in the style of another one. In this essay I answer the discussion questions of the sixth lecture in order. I group them into four parts: object detection, semantic segmentation, face recognition, and neural style transfer together with the extension of convolutions to 1D and 3D data.

Part 1. Object Detection

1. What is object localization and how?

Image classification only says which class is in the picture. Object localization (more precisely, classification with localization) also says where the object is by drawing a bounding box around it. To do this, the output layer of the convnet is extended. In addition to the class scores, the network outputs four numbers b_x, b_y, b_h, b_w: the center of the box and its height and width, measured relative to the image, whose top-left corner is (0, 0) and bottom-right corner is (1, 1). The target label therefore becomes a vector such as

$$y = [\, p_c,\ b_x,\ b_y,\ b_h,\ b_w,\ c_1,\ c_2,\ c_3 \,]$$

where p_c indicates whether any object is present and c_1, c_2, c_3 are the class indicators (for example pedestrian, car, motorcycle). If $p_c = 1$, the loss is computed over all components; if $p_c = 0$, the remaining components are "don't care" and only the error on p_c is counted. In the lecture the squared error is used for simplicity, although in practice one can use a logistic loss for p_c, squared error for the box, and a softmax loss for the classes.

2. How to detect landmarks?

Landmark detection uses the same idea but, instead of a box, the network outputs the (x, y) coordinates of important points. For a face we may define 64 landmarks such as the corners of the eyes, the edge of the mouth and the jaw line, so the final layer outputs $1 + 2 \times 64 = 129$ numbers: one for "is there a face?" and two coordinates per landmark. The training set must be labeled by people, and the identity of each landmark must be consistent across all images (landmark 1 is always the left corner of the left eye, and so on). Landmarks are the basic building block for emotion recognition, augmented-reality filters, and human pose estimation, where the landmarks are joints such as the shoulder, elbow and wrist.

3. What is object detection and how?

Object detection means finding all objects in an image, possibly many objects of several classes, and giving a bounding box for each one. The classical approach is the sliding windows detection algorithm. First, a convnet is trained on closely cropped images, where the object fills almost the whole picture, to

say whether the crop contains a car or not. Then, at test time, a small window is slid across the whole image with some stride, and every cropped region is passed through the convnet. The process is repeated with larger windows. The weakness is the computational cost: a fine stride and many window sizes mean a huge number of crops, each processed independently, while a coarse stride hurts the localization accuracy.

4. How to implement the sliding windows object detection algorithm convolutionally?

The key observation is that fully connected layers can be rewritten as convolutional layers. If the last volume before the fully connected layer is $5 \times 5 \times 16$, a fully connected layer with 400 units is mathematically the same as a convolution with 400 filters of size 5×5, which gives a $1 \times 1 \times 400$ volume; the next fully connected layers become 1×1 convolutions. Once the whole network is convolutional, we do not have to crop the image at all. Suppose the network was trained on $14 \times 14 \times 3$ inputs. If we feed a $16 \times 16 \times 3$ image, the output is not $1 \times 1 \times 4$ but $2 \times 2 \times 4$, and each of the four positions equals the result of running the original network on one of the four 14×14 windows. All windows are evaluated in a single forward pass, and the computation in the overlapping regions is shared, which makes the algorithm far more efficient. The remaining problem is that the window positions are fixed by the stride of the network, so the boxes are not very accurate.

5. How to predict the bounding box of an object?

The YOLO ("You Only Look Once") idea solves this. A grid, for example 3×3 in the lecture and 19×19 in practice, is placed on the image, and the localization label of Question 1 is defined for every grid cell. Each object is assigned to the single cell that contains its midpoint. With three classes the target volume is $3 \times 3 \times 8$, and the convnet is designed so that its output has exactly this shape. The box is specified relative to the cell: b_x and b_y are between 0 and 1 because the midpoint lies inside the cell, while b_h and b_w are expressed as a fraction of the cell size and can be larger than 1 when the object spans several cells. Because the network directly regresses these numbers, the box can have any aspect ratio and precise coordinates, and because it is one convolutional pass, it is fast enough for real-time detection.

6. What is intersection over union and why?

Intersection over union (IoU) measures how much two bounding boxes overlap. It is the area of their intersection divided by the area of their union, so it is 1 when the boxes are identical and 0 when they do not overlap. We need it for two reasons. First, it is the standard way to evaluate localization: by convention a predicted box is judged "correct" if its IoU with the ground-truth box is at least 0.5 (a stricter threshold such as 0.6 or 0.7 may be used). Second, it is a general similarity measure between boxes, which is used inside non-max suppression and when assigning objects to anchor boxes.

7. What is non-max suppression and why?

With a fine grid, several neighboring cells may all believe that they contain the midpoint of the same car, so the same object is detected several times. Non-max suppression (NMS) cleans this up so that each object is reported only once. First, all boxes with a low probability (for example $p_c \leq 0.6$) are discarded. Then, while boxes remain, the box with the highest p_c is selected as a final prediction, and every remaining box that has a high IoU (for example ≥ 0.5) with it is suppressed. In other words, we keep the maximum and suppress the non-maximal boxes that overlap with it. When there are several classes, NMS is carried out independently for each class.

8. What are anchor boxes and why?

So far each grid cell can detect only one object. If the midpoints of a pedestrian and a car fall into the same cell, the network cannot output both. Anchor boxes fix this. We define several box shapes in advance, for example a tall thin box and a wide flat box, and the label of each cell is repeated once per anchor: with two anchors the target becomes $3 \times 3 \times 16$. Each object is then assigned not only to the cell that contains its midpoint but also to the anchor box whose shape has the highest IoU with the object's box. Besides handling overlapping objects, anchors let different output units specialize in different shapes (tall objects such as pedestrians versus wide objects such as cars). Anchor shapes can be chosen by hand or by running k-means clustering on the box shapes of the training set. The method still fails in rare cases, such as three objects in one cell with only two anchors, or two objects with the same anchor shape in the same cell.

9. What is the YOLO object detection algorithm?

YOLO puts all of the above components together. For training, each image is labeled with a volume of shape (grid height) × (grid width) × (number of anchors) × (5 + number of classes); for example $19 \times 19 \times 5 \times 85$ for five anchors and 80 classes. A convnet maps the input image directly to this volume and is trained end to end. For prediction, the image is passed through the network once, so each cell produces one box per anchor. Boxes with a low score are removed, and then non-max suppression is run separately for each class to obtain the final detections. Since localization, classification and the sliding-window computation are all done by one convolutional network in one pass, YOLO is both accurate and very fast, which is why it is used in the "Car detection with YOLO" programming assignment.

Part 2. Semantic Segmentation

10. What is semantic segmentation and how?

Object detection draws rectangles, but some applications need to know the exact outline. Semantic segmentation labels every single pixel with a class. A self-driving car, for example, wants to know exactly which pixels are drivable road, and in medical imaging a segmentation of a chest X-ray or a brain MRI shows precisely where an organ or a tumor is, which helps diagnosis and surgical planning. The output is therefore not a class label or a few coordinates but a whole matrix of labels with the same height and width as the input. An ordinary convnet steadily shrinks the spatial size, so a segmentation network adds a second half that makes the volume grow again: the height and width are gradually increased back to the original size while the number of channels decreases. The final layer outputs $h \times w \times n_{classes}$ scores, and each pixel takes the class with the highest score.

11. What is transpose convolution and how?

The transpose convolution is the layer that makes the volume bigger; it is a learnable upsampling operation. In a normal convolution the filter is placed on the input and produces one number of the output. In a transpose convolution the filter is placed on the output. Take the lecture example of a 2×2 input, a 3×3 filter, padding $p = 1$ and stride $s = 2$, which gives a 4×4 output. Each input value multiplies the entire filter, and the scaled 3×3 copy is pasted into the output at a position that moves by the stride for each input element. Where the pasted copies overlap, the values are added, and the values that land in the padding border are ignored. Because the filter weights are learned, the network learns how best to upsample.

12. What is U-Net?

U-Net is the best-known architecture for semantic segmentation, and its diagram looks like the letter U. The left half is a normal convnet: blocks of convolutions with ReLU followed by max pooling, which reduce height and width and increase the number of channels. The right half uses transpose convolutions to bring the height and width back up. The crucial ingredient is the skip connections: the activations of each early, high-resolution layer are copied and concatenated to the corresponding layer of the right half. The decoder then has two kinds of information at once: high-level, contextual information from the deep layers ("there is a cat roughly in this region") and fine, low-level spatial detail from the early layers ("exactly which pixels look like fur"). A final 1×1 convolution maps the last volume to $h \times w \times n_{classes}$.

Part 3. Face Recognition

13. What is face recognition and how?

Two related problems must be distinguished. Face verification is a 1:1 problem: given an image and a claimed identity, decide whether the image is that person. Face recognition is a 1:K problem: given an image and a database of K people, output the identity of the person, or "not recognized". Recognition is harder, because a verification error is multiplied by the K comparisons, so the verification component must be extremely accurate. Both are solved by learning a similarity function d(img1, img2) that returns a small number when the two images show the same person and a large number otherwise. If d is below a threshold τ, the system declares "same person". For recognition, the input image is compared with every person in the database.

14. What is one-shot learning?

One-shot learning means recognizing a person after seeing just one example of that person's face, which is the usual situation in practice because a company has perhaps a single photo of each employee. Training a convnet with a softmax output over all employees does not work: one image per class is far too little data, and the network would have to be retrained whenever a new person joins. Learning the similarity function d instead solves both problems. The network learns in advance, from a large separate data set, how to compare two faces, so adding a new person only requires adding one picture to the database.

15. What is a Siamese network?

A Siamese network is the architecture used to implement d. The same convnet, with the same parameters, is applied to two different input images. Instead of ending in a softmax layer, the network ends in a vector of, say, 128 numbers, f(x), which is called the encoding (or embedding) of the image. The distance between two images is then defined as the squared norm of the difference between their encodings:

$$d(x^{(1)}, x^{(2)}) = || f(x^{(1)}) - f(x^{(2)}) ||^2$$

The parameters are learned so that this distance is small when $x^{(1)}$ and $x^{(2)}$ are the same person and large when they are different people.

16. What is the triplet loss function and why?

The triplet loss is one way to train such encodings. Each training example is a triplet of images: an anchor A, a positive P showing the same person as the anchor, and a negative N showing a different person. We want the anchor to be closer to the positive than to the negative by at least a margin α, and the loss is

$$L(A, P, N) = max(|| f(A) - f(P) ||^2 - || f(A) - f(N) ||^2 + \alpha, 0)$$

The loss is zero when the requirement is already satisfied and positive otherwise, and the cost is the sum over all triplets. The margin α is necessary because, without it, the network could satisfy the inequality trivially by outputting the same encoding (for example all zeros) for every image. The choice of triplets also matters: randomly chosen negatives are usually so different from the anchor that the constraint is satisfied without any learning, so training should focus on "hard" triplets where d(A, P) is close to d(A, N). Note that training requires several pictures of the same person, even though the deployed system needs only one picture per person.

17. How to use binary classification for face recognition?

An alternative to the triplet loss is to treat verification as binary classification. A Siamese pair of networks computes the encodings of two images, and these are fed to a logistic regression unit that outputs 1 if the two images are the same person and 0 if they are different. The input features are the element-wise differences between the encodings:

$$\hat{y} = \sigma(\Sigma_k w_k | f(x^{(i)})_k - f(x^{(j)})_k | + b)$$

The absolute difference can be replaced by other measures such as the chi-square similarity. The training set consists of pairs of images labeled "same" or "different", and the network is trained with ordinary backpropagation. A useful practical trick is to precompute and store the encodings of all people in the database, so that at test time only the new image has to be passed through the convnet.

Part 4. Neural Style Transfer and Beyond 2D

18. What is neural style transfer?

Neural style transfer takes a content image C (for example a photo of a campus) and a style image S (for example a painting by Van Gogh) and generates a new image G that shows the content of C drawn in the style of S. It is different from everything above in one important way: we do not train the weights of a network. The network is a fixed, pre-trained convnet such as VGG, and what we optimize are the pixels of the generated image itself.

19. What are deep convnets really learning?

To understand why this works, we need to know what the layers of a convnet represent. Following the visualization of Zeiler and Fergus, we can pick a hidden unit and find the image patches in the training set that maximize its activation. Units in the first layer respond to simple features such as edges at a particular angle or a particular color. Units in the second layer see a larger part of the image and respond to textures and simple shapes such as vertical stripes or round patterns. In deeper layers the receptive field keeps growing and the units detect more and more complex things: wheels, legs, honeycomb textures, text, and finally whole objects such as dogs or people. So the shallow layers capture low-level detail and the deep layers capture high-level content, and style transfer makes use of both.

20. How to define a cost function for neural style transfer?

The cost function measures how good a generated image is, and it has two parts:

$$J(G) = \alpha\, J_{content}(C, G) + \beta\, J_{style}(S, G)$$

The first term measures how similar the content of G is to the content of C, the second measures how similar the style of G is to the style of S, and the hyperparameters α and β set the relative weight of the two. The algorithm initializes G randomly (for example as a 100 × 100 × 3 noise image) and then runs gradient descent on its pixels, G := G − ∂J(G)/∂G, so that the noise is gradually turned into the content image rendered in the chosen style.

21. What is the content cost function?

We choose one hidden layer l of the pre-trained network to compute the content cost. If l is very shallow, G is forced to match C almost pixel by pixel; if it is very deep, the constraint is only something like "if there is a dog in C, there is a dog somewhere in G". A layer in the middle is therefore normally chosen. Let $a^{[l](C)}$ and $a^{[l](G)}$ be the activations of layer l for the two images. If these activations are similar, the two images have similar content, so

$$J_{content}(C, G) = \tfrac{1}{2} \,||\, a^{[l](C)} - a^{[l](G)} \,||^2$$

which is simply the element-wise sum of squared differences between the two activation volumes.

22. What is the style cost function?

Style is defined as the correlation between the activations of different channels in a layer. Each channel detects some feature, for example vertical texture or an orange tint. If two channels are highly correlated, then wherever the image has that texture it also tends to have that tint; this kind of co-occurrence is what we perceive as style. The correlations are collected in the style matrix (Gram matrix) of size $n_c \times n_c$:

$$G^{[l]}_{kk'} = \Sigma_i \Sigma_j\, a^{[l]}_{ijk}\, a^{[l]}_{ijk'}$$

where i and j run over the height and width and k, k′ index the channels. This matrix is computed for both the style image and the generated image, and the style cost of layer l is the normalized squared Frobenius distance between them:

$$J^{[l]}_{style}(S, G) = 1 / (2\, n_H\, n_W\, n_C)^2 \cdot \Sigma_k \Sigma_{k'} \left(G^{[l](S)}_{kk'} - G^{[l](G)}_{kk'} \right)^2$$

The results look better when several layers are used, so the overall style cost is a weighted sum $J_{style}(S, G) = \Sigma_l\, \lambda^{[l]}\, J^{[l]}_{style}(S, G)$, which takes into account both the low-level features of early layers and the high-level features of later layers.

23. How to generalize the convnets you learned for 2D data to 1D and 3D data?

The convolution idea is not limited to images. For 1D data such as an ECG signal, which is a sequence of voltage values over time, a 1D filter slides along a single axis: a 14-dimensional input convolved with a filter of length 5 gives a 10-dimensional output, and with 16 filters the output is 10 × 16. The next layer then uses filters of size 5 × 16, because the number of channels must match exactly as in 2D. For 3D data such as a CT scan, which is a stack of slices through the body, the volume has height, width and depth, and a 3D filter slides along all three axes: a 14 × 14 × 14 × 1 input convolved with 16 filters of size 5 × 5 × 5 gives a 10 × 10 × 10 × 16 output. Video can be treated in the same way, with time as the third dimension, to recognize motion or actions. The same output-size formula, parameter sharing

and sparsity of connections apply in every case; only the number of axes along which the filter moves changes.

Conclusion

These two weeks show how flexible the convolutional network is once the output and the loss are designed carefully. By changing the output to include box coordinates, a classifier becomes a detector, and with grids, anchor boxes, IoU and non-max suppression it becomes YOLO. By adding transpose convolutions and skip connections it becomes a U-Net that labels every pixel. By replacing the softmax with an encoding and training with the triplet loss or a binary classifier, it can recognize a face from a single example. And by fixing the weights and optimizing the image instead, the features it has learned can even be used to create art. The common lesson I take from this lecture is that the architecture stays largely the same, and what we ask the network to predict and how we measure its error determine what it can do.